> ArUco

square containing a binary black-and-white pattern

pattern provides:

- unique ID
- recognizable orientation → openCV can distinguish top-left from bottom-right even marker rotates
- 4 ordered outer corners

EXAMPLE: marker with 50mm side length

CORNERS: (-25, +25, 0) (+25, +25, 0) (-25, -25, 0) (+25, -25, 0)

=> Pose

pose contains two parts:

pose = rotation + translation → where the marker's center is in the cam. coord. frame

rotation (R or rvec) — wheter marker is frontal, tilted, turned sideways, upside down

translation (tvec)

tvec = [tx, ty, tz]

chain of finding:

```
Camera image
    ↓
Detect marker pattern and ID
    ↓
Detect its four pixel corners
    ↓
Use known physical corners + K + distortion
    ↓
Find the R,t that best explains those pixels
    ↓
Draw 3D axes or report distance
```

PnP: Perspective-n-Point
Inputs: detected comp.
Outputs: rvec, tvec.

PnP answers:

> "What rotation and translation would project these known physical corners onto these observed pixel corners?"

> "Find the marker position and rotation that would make its known 3D corners appear at these detected pixels."

1. If the marker moves farther away, what happens to its pixel size and `tz`?
2. If it moves toward the camera's right without changing distance, which part of `tvec` changes?
3. Why can't the camera calculate distance in millimetres if we don't tell it the marker's real physical size?

1. decrease, tz increase
2. tx
3. pixels reveal only size-to-distance ratio

→ 50mm marker at 500mm vs. 100mm marker at 1000mm looks same

⇒ Detection vs. Pose

> ArUco detection

↳ detector examines the image,
returns: - marker ID
- four ordered pixel corners

> PnP pose estimation

↳ matches those pixels with known physical corners:

Physical corner ↔ Detected pixel corner

then uses K
to calculate tvec, rvec

! corner order matters

1. Does ArUco detection itself produce distance in millimetres?

2. Does the marker ID tell us its physical printed size?

3. Which runs first: marker detection or PnP?

1. no, PnP needs to match the pxs w/ physical corners
2. no, which binary marker
3. marker detection → PnP

> ArUco Dictionaries & IDs

predefined collection of valid binary patterns

DICT_4X4_50

- 4X4 → internal binary pattern contains 4x4 grid
- 50 → contains 50 distinct marker IDs

The detector uses the pattern to determine:

- "This is marker 7."
- "This is its orientation."
- "These are its four ordered pixel corners."

! marker ID doesn't change only its physical scale does.

then PnP told;

marker side length = ——— mm → controls the metric scale of tvec

1. In `DICT_4X4_50`, what does `4X4` describe?
2. If marker ID 7 is printed twice as large, does its ID change?
3. Suppose the real marker is 100 mm wide, but we incorrectly tell PnP it is 50 mm. Would the estimated translation distances become approximately twice the real values or half the real values?

1. 4x4 grid, contained
2. no only the scale changes
3. goes half.

⇒ Reading the Translation Vector

marker origin at its center ; $tvec = [t_x, t_y, t_z]$

t_x → + right

t_y → + down

t_z → + forward depth

"the marker center's pos. measured from the camera"

> distance = $\sqrt{t_x^2 + t_y^2 + t_z^2}$

↳ if the marker is almost centered

$t_x, t_y \approx 0$

⇒ distance ≈ t_z

1. For `tvec = [0, 0, 600]` mm, what is the total distance?
2. For `tvec = [300, 0, 400]` mm, what are `tz` and the total distance?
3. If the marker moves upward in the image, should `ty` become more positive or more negative?

1. dist = t_z = 600mm
2. dist = 500mm
3. more negative.

⇒ Reading the Rotation Vector

Remember! t : where the marker center is

R : which way the marker is facing

each chessboard photo had:

- one tvec (position)
- one rvec (orientation)

$rvec = [r_x, r_y, r_z]$

However, these are not simply three independent angles like "rotate X by this, Y by that, Z by that."

OpenCV represents rotation using **axis-angle form**:

- The vector's direction describes the rotation axis
- The vector's length describes the rotation angle in radians

$|rvec| = \pi/2$ radians $= 90^\circ$

1. If you slide the marker right without tilting it, which should mainly change: `rvec` or `tvec`?
2. If you rotate the marker around its center without moving that center, which should mainly change?
3. If the magnitude of `rvec` is zero, how many degrees of rotation does it represent?

1. tvec, t_x increases
2. rvec
3. 0° rotation

⇒ How four Pixels Reveal Pose

> PnP examines the marker's exp.:

- marker shifts sideways ! clue abt. t_x
- marker shifts vertically ! clue abt. t_y
- marker becomes larg/small ! clue abt. t_z

For example:

Ex:

```
Centered square → move it farther → smaller centered square
```

Its orientation barely changed, so `rvec` stays similar while `tz` increases.

Ex:

```
Keep center still → tilt one side away → square becomes trapezoidal
```

Its position remains similar, but `rvec` changes.

1. The marker remains centered but becomes smaller. Which value most likely increased?
2. Its center and approximate size remain similar, but the square becomes strongly skewed. Is that mainly translation or rotation?
3. The marker moves right and becomes smaller. Which two translation components most likely increased?

1. t_z
2. rotation, rvec
3. $t_x \uparrow$, $t_z \uparrow$

⇒ Pose is Always relative to Something

PnP result will be:

"the marker's pose relative to cam."

tvec: [100, 0, 600] mm

↳ 100 mm to cam's right

↳ 600 mm infront of rt

→ another cam. viewing the same marker would calc. diff. tvec.

This also means two physical actions can create the same relative change:

- Move the marker left while keeping the camera still
- Move the camera right while keeping the marker still

From the camera's viewpoint, the marker moves left in both cases.

marker frame → camera frame

1. The camera moves closer to a stationary marker. What happens to the estimated `tz` ?
2. The camera moves right while the marker stays still. From the camera's viewpoint, which direction does the marker's `tx` move?
3. Can one camera-to-marker `tvec` alone tell us the marker's universal position in a room?

1. $tz \downarrow$
2. $tx \downarrow$; goes left
3. no, it depends on the reference

⇒ What will control Our Pose Accuracy?

PnP depends on the measurements two essential inputs:

K + distortion.

For reliable ArUco pose estimation, we need:

- The same camera
- The same `1920×1080` resolution
- The correct saved `K` and distortion
- The correct physical marker side length
- A flat marker
- Four clearly visible, sharp corners

- Wrong marker size → `tvec` gets the wrong physical scale
- Wrong `K` or image resolution → projection geometry becomes inaccurate
- Bent marker → its four corners no longer match our flat `Z=0` model
- Blurry or partially hidden corners → pose becomes noisy or fails
- Marker extremely far away → too few pixels for accurate corner detection
- Marker viewed almost edge-on → corner geometry becomes difficult to estimate

! good mathematics cannot completely rescue incorrect physical measurements

1. Can we directly use our `1920×1080` camera matrix on a `1280×720` camera frame without adjusting it?
2. If detection is perfect but we enter half the real marker size, which result is mainly incorrectly scaled?

3. If the detected corners jump slightly between frames because of blur, what do you expect `rvec` and `tvec` to do?

2. no we have to stick to the same baseline as calibration
2. tvec gets the wrong physical scale
3. rotation and translation might change so are tvec, rvec

- Marker ID tells us **identity**, not physical size.
- Detection alone produces an ID and four pixel corners—not millimetres.
- Printing and measuring the marker as 60 mm supplied the metric scale.
- PnP connected physical points, pixel points, and camera calibration.
- `tvec = [tx,ty,tz]` located the marker center relative to the camera.
- Total distance is `√(tx² + ty² + tz²)`, not always just `tz`.
- `rvec` represented marker orientation; the axes made it visually understandable.
- Pose was relative to the camera, not a universal room position.
- Rotation, translation, distance, and axis behaviour all matched your predictions.
- Occlusion caused detection failure because the complete marker and corners were unavailable.
- Small pose fluctuations occurred because every frame was solved independently without smoothing.

1. What information does ArUco detection produce before PnP?
2. Why did we need the marker's measured 60 mm size?
3. What four main inputs did PnP use in our experiment?
4. Interpret `tvec = [40, -20, 500]` mm.
5. If the marker remains stationary but its detected corners jitter, why do `rvec` and `tvec` also jitter?

1. marker ID + 4 ordered pixel corners
2. 60mm gives the real world scale. w/out it PnP cannot distinguish "small and close" from "large and farther"
3. physical marker corners
   detected marker corners
   camera matrix K
   distortion coefficients
4. 40 right, 20 up, 500 from camera
5. blur, hand motion, lighting affects inputs so does to rvec, tvec

calibration

known chessboard geo.
+ detected chessboard pixels
+ multiple photos

→ cam. matrix K
+ distortion coeffs

ArUco detection

camera image

→ marker ID
+ four ordered corner pixels

PnP

known marker geo
+ four detected pixel corners
+ distortion
+ K

→ rvec
+ tvec.